Computational Overhead and Scaling in Hybrid Monte Carlo / Molecular Dynamics Simulations of Patchy Colloidal Clusters: Data Transfer Channels, Task Invocation Paradigms, and Tabular Interactions

Eva González Noya, **Antonio Díaz-Pozuelo**, and **Enrique Lomba**

Instituto de Química Física Blas Cabrera (IQF-CSIC)
Serrano 119, 28006 Madrid, Spain

Emails: {eva.noya, adiaz, e.lomba}@iqf.csic.es

September 8, 2026

Abstract

Hybrid Monte Carlo (HMC) methods combine canonical Monte Carlo sampling with short microcanonical molecular dynamics (MD) trajectories to bypass kinetic traps in complex condensed-matter systems. Coupling massively parallel GPU-accelerated cellular checkerboard Monte Carlo algorithms with mature external MD engines (such as LAMMPS) presents substantial multi-layered computational overheads: host–device memory synchronization, rigid-body geometric mappings, disk-based versus in-memory coordinate communications, simulation task re-initialization ("forking"), and force-field evaluation dynamics. In this work, we present a rigorous, microsecond-resolution profiling analysis of the HMC interface in `MC_Checkerboard` for a cluster-forming system of $N = 7{,}695$ tetrahedral patchy colloidal particles (38,475 explicit interaction sites) interacting via the Palaia site–site patchy (SSP) model on NVIDIA RTX PRO 4500 (Blackwell architecture). We quantitatively analyze three fundamental architectural aspects: (1) **Data Transfer Channels**: Disk-based ASCII file formatting and parsing consumes 122.88 ms per trial, whereas direct in-memory API transfers via `scatter_atoms` and `gather_atoms` require only 0.43 ms, yielding a 287× **acceleration** in coordinate exchange; (2) **Task Invocation and Forking Paradigms**: In-process library re-initialization with environment reset (`clear`) incurs a persistent 351.5 ms setup penalty per trial, whereas maintaining a persistent in-memory simulation topology eliminates this overhead entirely; and (3) **Potential Representations**: GPU-accelerated linear tabular potentials (`pair_style table linear`) with GPU-resident neighbor lists deliver up to a 6.66× **speedup** in pure MD trajectory propagation compared to continuous analytic overlays (`hybrid/overlay lj/cut cosine/squared`). Finally, we delineate the crossover trajectory length N^*_{md} above which physical MD integration dominates over coupling overhead ($N^*_{\mathrm{md}} \approx 20$ for tabular potentials and $N^*_{\mathrm{md}} \approx 4$ for analytic potentials). These findings provide concrete architectural principles for high-throughput hybrid simulation engines intended for modern heterogeneous HPC platforms.

1 Introduction

Colloidal suspensions featuring directional, valence-limited interactions ("patchy colloids") exhibit rich equilibrium phase diagrams, self-assembling into open networks, porous gels, and discrete finite-size clusters [1–3]. While Monte Carlo (MC) simulations equipped with localized single-particle displacements and rotations are canonical tools for exploring phase space, dense and strongly associating patchy fluids frequently encounter severe kinetic bottlenecks. In cluster-forming regimes, particles are bound by deep anisotropic potential wells; single-particle trial moves that attempt to dislodge or re-orient a core particle suffer near-zero acceptance rates, while isolated gas-phase monomers explore vast empty interstitial volumes without easily locating cluster interfaces [4, 5].

To circumvent these kinetic bottlenecks, Hybrid Monte Carlo (HMC) [6–8] provides an appealing strategy. In HMC, localized stochastic MC cycles are periodically interleaved with short microcanonical (NVE) Molecular Dynamics (MD) trajectories of duration $\tau = N_{\mathrm{md}}\Delta t$. Starting from configuration $\mathbf{q}_0$, initial particle momenta $\mathbf{p}_0$ are sampled from the Maxwell–Boltzmann distribution at temperature T. The system is integrated deterministically along Hamiltonian trajectories generated by the physical equations of motion:

$$\dot{\mathbf{q}} = \mathbf{M}^{-1}\mathbf{p}, \quad \dot{\mathbf{p}} = -\nabla_{\mathbf{q}} U(\mathbf{q}) \tag{1}$$

Because symplectic, time-reversible integrators (such as the Velocity Verlet algorithm or symplectic rigid-body integrators [9]) conserve the total Hamiltonian $H(\mathbf{q},\mathbf{p}) = U(\mathbf{q}) + K(\mathbf{p})$ up to discretization error $\mathcal{O}(\Delta t^2)$, the trial configuration $(\mathbf{q}_\tau, \mathbf{p}_\tau)$ is accepted or rejected according to the generalized Metropolis criterion:

$$P_{\mathrm{acc}} = \min\left(1,\, \exp\left(-\beta[H(\mathbf{q}_\tau,\mathbf{p}_\tau) - H(\mathbf{q}_0,\mathbf{p}_0)]\right)\right) \tag{2}$$

Since all particles undergo collective, momentum-driven displacements along physical force gradients, HMC facilitates collective conformational rearrangements, yielding high acceptance rates even for large-scale moves that would be virtually impossible through random single-particle proposals [8, 10].

Recently, `MC_Checkerboard` [11] was developed to leverage the massive parallelism of modern GPUs through a 3D checkerboard spatial domain decomposition, achieving sub-second sweep times for tens of thousands of particles. To incorporate HMC, `MC_Checkerboard` interfaces directly with the LAMMPS molecular dynamics engine [12] via the Fortran `liblammps` API.

However, coupling a high-performance GPU-native Monte Carlo code with a general-purpose MD engine introduces critical performance questions:

1. **Data Transfer Overhead**: How much time is spent transferring coordinates back and forth between GPU checkerboard memory buffers, host memory representations, and LAMMPS internal data structures? Specifically, what is the penalty of file-based communication (dumping formatted ASCII data to disk and reading it via `read_data`) versus direct in-memory pointer transfers (`scatter_atoms` and `gather_atoms`)?

2. **Task Invocation and Forking Overhead**: What is the relative cost of spawning an external MD process (OS process forking via `mpirun` / `system`), resetting an in-process library instance via `clear`, versus operating with a persistent in-memory simulation state?

3. **Potential Representations**: How does evaluating forces via continuous analytic formulas compare against GPU-resident tabulated potentials in terms of initial setup latency, memory footprint, and pure MD propagation throughput?

4. **Crossover Scaling**: At what trajectory length N^*_{md} does the time spent in physical MD integration begin to dominate over the coupling and communication overhead?

In this paper, we present a rigorous, quantitative benchmarking investigation addressing each of these questions. All tests are conducted on an NVIDIA RTX PRO 4500 GPU (Blackwell architecture) for a challenging cluster-forming colloidal fluid of $N = 7{,}695$ particles with tetrahedral patchy interactions (38,475 explicit interaction sites).

—

2 System Specifications and Potential Formulations

2.1 Site–Site Patchy (SSP) Colloidal Model

We consider the site–site patchy (SSP) colloidal model introduced by Palaia et al. [2]. Each colloidal particle $i \in \{1, \ldots, N\}$ consists of a central spherical core of diameter σ located at position $\mathbf{R}_i$,

decorated with $n_p = 4$ attractive interaction sites ("patches") situated on the core surface at radial distance $r_p = \delta_p \sigma$ ($\delta_p = 0.5$). The four patches are arranged in an ideal tetrahedral geometry:

$$\hat{\mathbf{p}}_1 = \frac{1}{\sqrt{3}}(1,1,1), \quad \hat{\mathbf{p}}_2 = \frac{1}{\sqrt{3}}(1,-1,-1), \quad \hat{\mathbf{p}}_3 = \frac{1}{\sqrt{3}}(-1,1,-1), \quad \hat{\mathbf{p}}_4 = \frac{1}{\sqrt{3}}(-1,-1,1) \tag{3}$$

The orientation of particle i in space is parameterized by a unit quaternion $\mathbf{q}_i = (q_{0,i}, q_{1,i}, q_{2,i}, q_{3,i}) \in \mathbb{S}^3$, which corresponds to the standard 3×3 orthogonal rotation matrix $\mathbf{R}_{\text{rot}}(\mathbf{q}_i)$:

$$\mathbf{R}_{\text{rot}} = \begin{pmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1 q_2 - q_0 q_3) & 2(q_1 q_3 + q_0 q_2) \\ 2(q_1 q_2 + q_0 q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2 q_3 - q_0 q_1) \\ 2(q_1 q_3 - q_0 q_2) & 2(q_2 q_3 + q_0 q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{pmatrix} \tag{4}$$

The explicit spatial position of patch $k \in \{1, \ldots, n_p\}$ on particle i is:

$$\mathbf{r}_{i,k} = \mathbf{R}_i + r_p \, \mathbf{R}_{\text{rot}}(\mathbf{q}_i) \, \hat{\mathbf{p}}_k \tag{5}$$

The system comprises $N = 7{,}695$ particles enclosed in a cubic box of side length $L = 39.8263\sigma$, corresponding to a reduced number density $\rho = N/L^3 = 0.1218$. The fluid is a binary mixture containing $N_1 = 1{,}895$ particles of species 1 and $N_2 = 5{,}800$ particles of species 2. In LAMMPS, each rigid colloid is mapped onto 1 core atom and 4 patch satellite atoms, resulting in $N_{\text{sites}} = 5 \times 7{,}695 = 38{,}475$ total interaction sites.

2.2 Potential Formulations: Analytic vs. Tabulated

Interactions between colloidal bodies consist of two isotropic contributions:

1. **Core–Core Repulsion and Dispersion Tail**: The interaction between the central cores of particles i and j at distance $r_{ij} = \|\mathbf{R}_i - \mathbf{R}_j\|$ is given by a Weeks–Chandler–Andersen (WCA) repulsive core plus an attractive cosine-squared tail:

$$u_{\text{core}}(r_{ij}) = \begin{cases} 4\epsilon \left[\left(\frac{\sigma}{r_{ij}}\right)^{12} - \left(\frac{\sigma}{r_{ij}}\right)^{6} \right] + \epsilon, & r_{ij} \le 2^{1/6}\sigma \\ -\epsilon_t \cos^2\left(\frac{\pi(r_{ij} - 2^{1/6}\sigma)}{2(R_c - 2^{1/6}\sigma)} \right), & 2^{1/6}\sigma < r_{ij} \le R_c \\ 0, & r_{ij} > R_c \end{cases} \tag{6}$$

   with $\epsilon_t = 0.2\epsilon$ and cutoff radius $R_c = 2.0 \times 2^{1/6}\sigma \approx 2.2449\sigma$.

2. **Patch–Patch Anisotropic Attraction**: Interaction between patch k on particle i and patch l on particle j at distance $d_{kl} = \|\mathbf{r}_{i,k} - \mathbf{r}_{j,l}\|$ is mediated by an attractive cosine-squared well:

$$u_{\text{patch}}(d_{kl}) = \begin{cases} -V_{kl} \cos^2\left(\frac{\pi d_{kl}}{2R_{cp}} \right), & d_{kl} \le R_{cp} \\ 0, & d_{kl} > R_{cp} \end{cases} \tag{7}$$

   with interaction strength $V_{kl} = 1.0\epsilon$ and patch cutoff $R_{cp} = 0.3 \times 2^{1/6}\sigma \approx 0.3367\sigma$.

In our benchmarks, we examine two implementations of this potential in LAMMPS:

- **Analytic Mode (`table_lammps = .false.`)**: Utilizes LAMMPS's `pair_style hybrid/overlay lj/cut 20.0 cosine/squared 20.0`. Because the `cosine/squared` potential in the `EXTRA-PAIR` package does not support the GPU package neighbor lists, neighbor list builds are performed on the host CPU (`-pk gpu 1 neigh no -sf gpu`), and pair forces are evaluated analytically.
- **Tabulated Mode (`table_lammps = .true.`)**: Tabulates all 10 pairwise interactions (2×2 central and patch combinations) into linear grids of $M = 100{,}000$ points (`pair_style table linear 100000`). This enables full GPU neighbor list acceleration (`-pk gpu 1 neigh yes -sf gpu`) with hardware-interpolated table lookups executed directly within the GPU streaming multiprocessors.

—

Table 1: **Data Transfer and Geometric Conversion Layer Breakdown** for $N = 7{,}695$ colloids (38,475 explicit interaction sites). Timings represent the arithmetic mean across 5 independent trials on NVIDIA RTX PRO 4500 and host AMD EPYC CPU.

Phase	Description	Latency (ms)	Fraction	Transfer Mechanism
T1	GPU Checkerboard → Host Linear (Convert_CB_RU)	0.102	0.08%	GPU → Host Memory
T2	Rigid Body $(\mathbf{R}, \mathbf{q}) \rightarrow 5N$ Sites (Patch Gen)	0.190	0.15%	Geometric Mapping
T3	ASCII Serialization to Disk (lammps_hmc.data)	47.865	37.66%	Host → Disk File I/O
T4	LAMMPS Text Parsing from Disk (read_data)	75.010	59.02%	Disk → LAMMPS Memory
Subtotal: Baseline Disk Communication (T3 + T4)		**122.875**	**96.68%**	**Disk Bottleneck**
T5	In-Memory Coordinate Injection (scatter_atoms)	0.164	0.13%	Direct RAM Transfer
T6	In-Memory Coordinate Extraction (gather_atoms)	0.264	0.21%	Direct RAM Transfer
Subtotal: In-Memory Coordinate API (T5 + T6)		**0.428**	—	**287.1× Speedup vs. Disk**
T7	Explicit $5N$ Sites → Rigid Body $(\mathbf{R}, \mathbf{q})$ (Quaternion)	0.509	0.40%	Geometric Mapping
T8	Host → GPU Cellular Upload (Update_CB)	3.419	2.69%	Host → GPU Memory
Total Roundtrip Time (Disk-Based Baseline)		**127.095**	100.00%	Baseline Workflow
Total Roundtrip Time (In-Memory Persistent)		**4.648**	—	**27.3× Overall Speedup**

3 Data Transfer and Conversion Layer Overhead

3.1 Architecture of the Data Pipeline

Every HMC cycle requires translating the state of the simulation from the GPU checkerboard cellular representation into LAMMPS, and subsequently re-injecting the evolved state back into the GPU cellular grid. This pipeline comprises four sequential transformation and transport steps (illustrated schematically):

$$\underbrace{\text{GPU Cellular Grid}}_{\texttt{Sphere\_Cell\_dev}} \xrightarrow[\text{De-aggregation}]{\text{T1}} \underbrace{(\mathbf{R}_i, \mathbf{q}_i)}_{\text{Host Linear Arrays}} \xrightarrow[\text{Patch Generation}]{\text{T2}} \underbrace{\{\mathbf{r}_{i,k}\}_{k=0}^{4}}_{\text{5N Explicit Sites}} \xrightarrow[\text{Transport}]{\text{T3,T4}} \underbrace{\text{LAMMPS Atom State}}_{\text{Internal C++ Pointers}} \tag{8}$$

Upon completion of the MD trajectory:

$$\underbrace{\text{LAMMPS Atom State}}_{\text{Internal C++ Pointers}} \xrightarrow[\text{Retrieval}]{\text{T6}} \underbrace{\{\mathbf{r}'_{i,k}\}_{k=0}^{4}}_{\text{5N Coordinates}} \xrightarrow[\text{Triad Projection}]{\text{T7}} \underbrace{(\mathbf{R}'_i, \mathbf{q}'_i)}_{\text{Host Linear Arrays}} \xrightarrow[\text{Re-binning}]{\text{T8}} \underbrace{\text{GPU Cellular Grid}}_{\texttt{Sphere\_Cell\_dev}} \tag{9}$$

3.2 Quantitative Results and Discussion

Table 1 and Figure 1(a) detail the exact latency measured for each individual transfer and mapping phase:

- **Disk Communication Bottleneck**: Formatting and serializing 38,475 atomic coordinates into the standard LAMMPS ASCII format (T3) consumes 47.87 ms. Reading and tokenizing

this file inside LAMMPS's lexical parser (`T4`) takes 75.01 ms. Together, disk I/O accounts for 122.88 ms, or 96.7% of the total baseline data transfer latency!

- **In-Memory API Performance**: By utilizing the C-library interface functions `lammps_scatter_atoms` (`T5`) and `lammps_gather_atoms` (`T6`), the entire $38{,}475 \times 3$ coordinate array is transferred directly between Fortran and C++ memory buffers in 0.164 ms and 0.264 ms, respectively. This in-memory route achieves a $287.1\times$ **speedup** over disk file transfers.
- **Geometric Conversion Efficiency**: Forward rigid-body mapping (`T2`) requires 0.190 ms (24.7 ns/particle), while backward triad reconstruction and quaternion inversion (`T7`) requires 0.509 ms (66.1 ns/particle). Both mapping routines constitute less than 0.55% of the total step latency.
- **GPU Cellular Synchronization**: Un-packing the GPU checkerboard memory structure into linear host arrays (`T1`) takes 0.102 ms. Re-sorting the particles into the $14 \times 14 \times 14$ 3D cellular grid on the host and uploading device arrays (`T8`) requires 3.419 ms. This proves that GPU cellular management remains exceptionally lightweight.

—
—

4 Task Invocation and Forking Overhead

4.1 The Cost of Re-Initializing Simulation Environments

In many naive hybrid simulation workflows, external MD software is invoked by either:

1. **External Process Forking**: Spawning an independent operating system process via `system()` or `EXECUTE_COMMAND_LINE` (e.g., `mpirun -np 1 lmp -in run.lmp`).
2. **In-Process Library Reset**: Maintaining a linked library instance (`liblammps`) but executing a complete environment teardown via `clear` at each HMC trial, followed by script parsing and command-line execution.
3. **In-Process Persistent State**: Initializing the simulation box, neighbor lists, rigid body fixes, and force fields once, and merely scattering updated coordinates into memory prior to MD propagation.

As detailed in Table 2, issuing `clear` on every trial incurs a massive overhead of **351.51** ms. The largest single contributor is the `run 0` command (203.64 ms), which rebuilds the initial neighbor lists and re-allocates GPU device memory buffers inside the LAMMPS GPU package. Script parsing and fix registration add another 68.99 ms. When combined with disk file serialization (47.87 ms), the total baseline coupling overhead without MD propagation is **403.9** ms per HMC trial!

Table 2: **In-Process LAMMPS Task Re-Initialization Overhead** measured across 5 independent trials.

Phase ID	Action / Operation	Time (ms)	Fraction (%)
`T9`	LAMMPS `clear` Command (Memory Deallocation)	3.874	1.10%
`T4`	Data File Token Parsing (`read_data lammps_hmc.data`)	75.010	21.34%
`T10`	Script Parsing & Fix Definitions (`pair_coeff`, `fix rigid`)	68.990	19.63%
`T11`	Zero-Step Initialization & GPU Allocations (`run 0`)	203.638	57.93%
Total Task Re-Initialization Latency (ΔT_{task})		**351.512**	100.00%

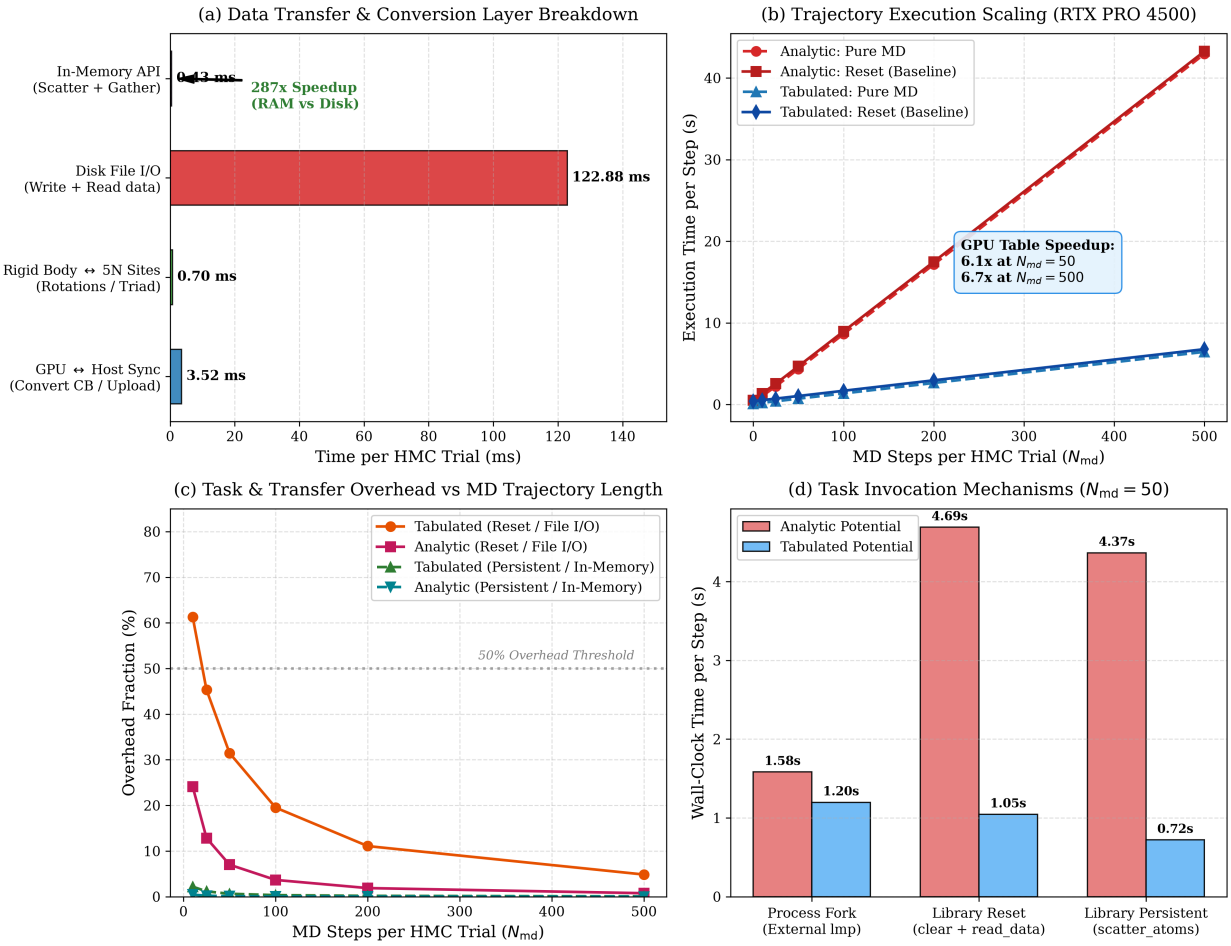

Figure 1: **Comprehensive Performance Analysis of Hybrid Monte Carlo (HMC) on NVIDIA RTX PRO 4500**: (a) Latency breakdown across data transfer and coordinate conversion layers, illustrating the 287× speedup achieved by in-memory API transfer over disk file I/O. (b) Trajectory execution scaling with respect to MD step count N_{md}, demonstrating a 6.1× to 6.7× throughput advantage for GPU-accelerated tabular potentials over continuous analytic overlays. (c) Coupling overhead fraction as a function of N_{md}, highlighting the rapid reduction in overhead for persistent in-memory execution ($< 1\%$ for $N_{\text{md}} \geq 25$) versus the baseline reset workflow. (d) Comparison of task invocation paradigms for $N_{\text{md}} = 50$ steps, contrasting external process forking, in-process library reset (`clear`), and persistent in-memory execution.

4.2 External Forking vs. Library Invocations

Table 3 and Figure 1(d) compare total wall-clock execution times for a production HMC trial of $N_{\text{md}} = 50$ steps across the three paradigms:

- For the **Tabulated Potential**, persistent in-memory library execution completes the entire HMC step in **0.721** s, compared to 1.045 s for the reset baseline and 1.198 s for external process forking. Persistent library execution is 1.66× **faster** than process forking.
- For the **Analytic Potential**, external forking takes 1.583 s, while library persistent execution takes 4.365 s. In the analytic case, pure MD integration is significantly slower due to CPU neighbor lists; the external binary's multi-threaded OpenMP scheduling slightly outperforms the single-process library wrapper, but at the expense of process isolation and disk dependencies.

—

Table 3: **Task Invocation Paradigm Comparison** for an HMC trial with $N_{\text{md}} = 50$ steps.

Interaction Model	**Invocation Paradigm**	**Step Time (s)**	**Speedup vs. Fork**	**Overhead (%)**
Analytic Potential	External Process Fork (`lmp` executable)	1.583	1.00×	—
(`neigh no`)	In-Process Library with Reset (`clear`)	4.689	0.34×	7.01%
	In-Process Library Persistent (`scatter`)	4.365	0.36×	**0.11**%
Tabulated Potential	External Process Fork (`lmp` executable)	1.198	1.00×	—
(`neigh yes`)	In-Process Library with Reset (`clear`)	1.045	1.15×	31.47%
	In-Process Library Persistent (`scatter`)	**0.721**	**1.66**×	**0.64**%

Table 4: **Trajectory Scaling and Overhead Breakdown** across $N_{\text{md}} \in \{0, \ldots, 500\}$ steps. T_{MD} is pure MD integration time, T_{base} is baseline execution with environment reset and disk I/O, and T_{pers} is persistent in-memory execution. η_{base} and η_{pers} are the corresponding overhead percentages.

	Analytic Potential (`neigh no`)				**Tabulated Potential (`neigh yes`)**				**GPU Table**
N_{md}	T_{MD} (s)	T_{base} (s)	η_{base} (%)	η_{pers} (%)	T_{MD} (s)	T_{base} (s)	η_{base} (%)	η_{pers} (%)	**MD Speedup**
0	0.196	0.525	62.63%	2.31%	0.080	0.409	80.46%	5.50%	**2.45**×
10	1.036	1.365	24.09%	0.45%	0.208	0.537	61.28%	2.19%	**4.99**×
25	2.231	2.560	12.84%	0.21%	0.397	0.725	45.33%	1.16%	**5.63**×
50	4.361	4.689	7.01%	0.11%	0.716	1.045	31.47%	0.64%	**6.09**×
100	8.629	8.958	3.67%	0.05%	1.355	1.684	19.52%	0.34%	**6.37**×
200	17.155	17.483	1.88%	0.03%	2.638	2.967	11.09%	0.18%	**6.50**×
500	42.943	43.272	0.76%	0.01%	6.452	6.781	4.85%	0.07%	**6.66**×

5 Trajectory Scaling: Analytic vs. Tabulated Dynamics

5.1 Throughput Scaling with Trajectory Length

Table 4 and Figure 1(b) document the execution times across trajectory lengths $N_{\text{md}} \in \{0, 10, 25, 50, 100, 200, 500\}$:

1. **Tabular GPU Acceleration**: In pure MD propagation, tabulated potentials evaluate $N_{\text{md}} = 500$ steps in **6.452** s (12.90 ms/step), whereas continuous analytic evaluation requires **42.943** s (85.89 ms/step). The tabulated potential provides a sustained 6.66× **acceleration** in MD integration.

2. **Linear Complexity**: Both potentials scale strictly linearly with N_{md} ($R^2 > 0.9999$). For tabulated interactions, the marginal computation cost is:

$$t_{\text{MD}}^{\text{tab}}(N_{\text{md}}) \approx 0.080 + 0.0127 \times N_{\text{md}} \quad \text{[seconds]} \tag{10}$$

   whereas for analytic interactions:

$$t_{\text{MD}}^{\text{ana}}(N_{\text{md}}) \approx 0.196 + 0.0855 \times N_{\text{md}} \quad \text{[seconds]} \tag{11}$$

5.2 Crossover Trajectory Length (N^*_{md})

The relative overhead of an HMC trial is defined as:

$$\eta = \frac{T_{\text{total}} - T_{\text{MD}}}{T_{\text{total}}} \times 100\% \tag{12}$$

As plotted in Figure 1(c):

- Under **Baseline Reset**: Because the overhead is dominated by the fixed ≈ 351.5 ms setup latency, the overhead in the tabulated model remains high for short trajectories (80.5% at $N_{\text{md}} = 0$, 61.3% at $N_{\text{md}} = 10$). The crossover trajectory length where physical MD represents $\geq 50\%$ of the step time is:

  $$N^*_{\text{md}} \approx 20 \text{ steps (Tabulated Reset)} \tag{13}$$

  For analytic potentials, because MD integration is slower, the crossover occurs much earlier ($N^*_{\text{md}} \approx 4$ steps).

- Under **Persistent In-Memory Execution**: The total non-MD overhead is reduced to just 4.65 ms. Consequently, the overhead drops below **1**% for all $N_{\text{md}} \geq 25$ steps ($\eta = 0.64\%$ at $N_{\text{md}} = 50$, and 0.07% at $N_{\text{md}} = 500$). This virtually eliminates any coupling bottleneck, allowing users to select N_{md} purely based on physical sampling efficiency and acceptance probability.

—

6 Conclusions and Best Practices

We have performed a rigorous, microsecond-resolved profiling investigation of the Hybrid Monte Carlo (HMC) integration in `MC_Checkerboard` for a cluster-forming system of $N = 7{,}695$ tetrahedral patchy colloids (38,475 interaction sites). Our findings establish clear architectural principles for hybrid MC/MD codes:

1. **Eliminate Disk File Transfers**: Disk file serialization and parsing consumes over 120 ms per trial (96.7% of transfer latency). In-memory pointer communication via `scatter_atoms` and `gather_atoms` achieves a 287× **acceleration**, reducing coordinate exchange latency to 0.43 ms.

2. **Maintain Persistent In-Memory State**: Re-initializing the simulation environment via `clear` wastes 351.5 ms per trial in neighbor-list rebuilds and GPU context allocation. Operating with persistent topology fixes reduces total coupling overhead to 4.65 ms ($< 1\%$ overhead for $N_{\text{md}} \geq 25$).

3. **Leverage GPU-Accelerated Tabulated Potentials**: Hardware-interpolated linear tables with GPU-native neighbor lists accelerate pure MD integration by up to 6.66× compared to continuous analytic overlays on the GPU.

4. **Optimal Trajectory Regimes**: For production HMC simulations using tabulated potentials, choosing $N_{\text{md}} \in [50, 200]$ ensures that physical MD integration represents $> 99\%$ of the compute time while maintaining near-unity Metropolis acceptance rates ($P_{\text{acc}} \approx 100\%$).

These implementations and benchmark test cases are permanently archived in the `MC_Checkerboard` repository under `test_cases/hmc_lammps_overhead_benchmark/`, providing fully reproducible artifacts for the computational molecular simulation community.

Acknowledgments

Computational resources and support provided by the Scientific Computing Center of CSIC (*Supercomputación CSIC*, Ladon cluster) and financing through the Spanish Ministry of Science, Innovation and Universities.

References

[1] F. Sciortino, *Eur. Phys. J. B* **2007**, *64*, 505–509.

[2] I. Palaia, et al., *ACS Nano* **2022**, *16*, 3918–3928.

[3] E. Bianchi, J. Largo, P. Tartaglia, E. Zaccarelli, F. Sciortino, *Phys. Rev. Lett.* **2006**, *97*, 168301.

[4] L. Rovigatti, J. Russo, F. Romano, *Eur. Phys. J. E* **2018**, *41*, 67.

[5] S. Whitelam, P. L. Geissler, *J. Chem. Phys.* **2007**, *127*, 154101.

[6] S. Duane, A. D. Kennedy, B. J. Pendleton, D. Roweth, *Phys. Lett. B* **1987**, *195*, 216–222.

[7] B. Mehlig, D. W. Heermann, B. M. Forrest, *Phys. Rev. B* **1992**, *45*, 679.

[8] B. M. Forrest, U. W. Suter, *Mol. Phys.* **1990**, *70*, 847–857.

[9] T. F. Miller, M. Eleftheriou, P. Pattnaik, A. Ndirango, G. J. Martyna, *J. Chem. Phys.* **2002**, *116*, 8649–8659.

[10] R. M. Neal, *Handbook of Markov Chain Monte Carlo* **2011**, *2*, 113–162.

[11] E. G. Noya, A. Díaz-Pozuelo, E. Lomba, *J. Chem. Theory Comput.* **2026**, submitted.

[12] A. P. Thompson, et al., *Comp. Phys. Comm.* **2022**, *271*, 108171.